

CAMPUSDESK

CampusDesk V2

Technical & Functional Documentation

Features, architecture, database, security, testing and deployment



Version 2.0 · October 2026

Cloud-based student management & campus productivity platform

React 18 · Vite 5 · Supabase (PostgreSQL, Auth, Storage, Realtime)

Roles: Student · Teacher · Parent · Administrator

Evolution of CampusDesk V1 on the same Supabase backend

Table of contents

1. Executive summary	5
1.1 What V2 delivers	5
1.2 At a glance	6
2. Background and goals	6
2.1 From V1 to V2	6
2.2 Problems V2 set out to fix	7
2.3 Design principles	7
3. Architecture	8
3.1 Layers inside the frontend	9
3.2 A request, end to end	9
4. Technology stack and project structure	10
4.1 Repository layout	10
5. Roles, navigation and routing	11
5.1 The shell	11
5.2 Route protection	11
6. Authentication and account management	12
6.1 The building blocks	12
6.2 Account states	12
6.3 The lifecycle	13
6.4 Registration screen	13
6.5 Account-state pages	13
6.6 Login decision tree	14
6.7 AuthContext and session handling	14
6.8 Administrator workflow: Account Requests	14
6.9 The review and provisioning dialog	15
6.10 Safety rules inside the provisioning functions	15
6.11 Password reset	16
6.12 Other ways accounts are created	16
6.13 Compatibility with existing V1 users	16
7. Dashboards	17
7.1 Student dashboard	17
7.2 Parent dashboard	17
7.3 Teacher dashboard	17
7.4 Admin dashboard	18

7.5 Implementation notes	18
8. Timetable	18
8.1 Weekly grid page	18
8.2 Time and conflict rules	19
9. Attendance	19
9.1 One definition everywhere	19
9.2 Student and parent view	19
9.3 Teacher marking workflow	19
9.4 Aggregates for dashboards	19
10. Fees and payments	20
10.1 The state machine	20
10.2 Database enforcement	20
10.3 Student and parent screens	21
10.4 Administrator fee console	21
10.5 Receipts	21
11. Exams, marks and results	22
11.1 Grades	22
11.2 Missing marks are not zero	22
11.3 Review before publishing	22
11.4 Who sees what	22
12. Academic Report	23
13. Other modules	24
13.1 Notifications	24
13.2 Global search	24
13.3 Notices and events	24
13.4 Leave	24
13.5 Teacher queries (chat)	24
13.6 Documents	24
13.7 Profile and settings	25
13.8 People management (administrators)	25
14. Design system and themes	25
14.1 Palette	25
14.2 Theme behaviour	26
14.3 Components and states	26
15. Security model	26

15.1 What a malicious user cannot do	27
15.2 Findings fixed in the existing database	27
15.3 Implementation details	27
16. Database reference	28
16.1 Migrations	28
16.2 Two install paths — never mixed	28
16.3 Main tables	29
17. Testing and quality	29
17.1 Test results delivered	30
17.2 What was found by testing	30
18. Setup, deployment and operations	30
18.1 Local development	30
18.2 Database setup	30
18.3 GitHub Pages deployment	31
18.4 First-time operation checklist	31
18.5 Optional: Edge Function	31
19. Demo data (fresh database only)	31
20. Known limitations and future work	32
20.1 Not implemented	32
20.2 Behaviour to be aware of	32
20.3 Suggested next steps	33
Appendix A — Database function reference	34
Appendix B — Important source files	35

1. Executive summary

CampusDesk V2 is a web application for running the day-to-day life of a school or college: who the students, teachers and parents are, which class meets when, who attended, how everyone performed, what fees are due and paid, and how the institution talks to its community. It is the second generation of CampusDesk. V1 was a collection of pages; V2 is organised around a simple idea — **navigation belongs in the sidebar, information belongs on the dashboard** — so every person sees a personalised overview of what matters to them today, and goes to a dedicated page only when they need to do something.

V2 runs entirely in the browser as a static site (hosted on GitHub Pages or Vercel) and uses **Supabase** as its backend: PostgreSQL for data, Supabase Auth for sign-in, Storage for files and Realtime for live notifications. There is no custom server. Because there is no server code to hide behind, every rule that matters — who may see which row, how fees are verified, when results become visible, who may become an administrator — is enforced **inside the database** with Row Level Security, constraints, triggers and functions. The React code makes the experience pleasant; the database makes it safe.

1.1 What V2 delivers

- **Four role-specific experiences** — Student, Teacher, Parent (with a multi-child switcher) and Administrator — each with its own sidebar, dashboard and permissions.
- **A complete account lifecycle** — public registration creates a *pending* account; an administrator reviews it, assigns the real role, provisions the student/parent/teacher record and activates it. Nobody can choose or grant themselves a role from the browser.
- **Personalised dashboards** — today's timetable with *current* and *next* class, an Action-required panel, KPIs, charts, notices and events; for administrators, institution-wide analytics and a System Health panel.
- **A professional Academic Report** — student profile, student ID, class teacher, guardian, subject-wise attendance, exam-wise marks, summary indicators, printable as a clean PDF.
- **A fee system that cannot be cheated or double-charged** — a database-enforced state machine (due → pending verification → partial / paid / rejected), verification queue, official receipts only for verified payments.
- **Safe result publication** — missing marks are never converted to zero; administrators review a summary (evaluated vs incomplete) before publishing.
- **One attendance definition everywhere** — dashboards, reports and analytics always agree.
- **Light, dark and match-device themes** with a soft, multi-colour palette, checked for accessible contrast.
- **Automated tests** — 66 unit/UI tests, 173 database assertions (including concurrency) and a service-level lifecycle check.

1.2 At a glance

Item	Value
Frontend	React 18.3, Vite 5, React Router 6 (HashRouter), Recharts 2, supabase-js 2 (PKCE)
Backend	Supabase: PostgreSQL + Auth + Storage + Realtime (+ one optional Edge Function)
Source size	121 source files, $\approx$ 10,500 lines of JavaScript/JSX in src/; $\approx$ 4,500 lines of SQL; $\approx$ 850 lines of tests
Database	Migrations 001-008 (additive, safe for an existing V1 database) + a separate fresh-install path
Routes	$\approx$ 40 routes, each guarded by role and account state
Tests	66 unit/UI + 173 database assertions + 28 service-level checks (see section 17)
Deployment	Static build; GitHub Actions workflow for GitHub Pages; works on Vercel; env vars injected at build time

How to read this document

Sections 2–4 explain the background and architecture. Sections 5–13 describe each feature and how it is implemented (screens, services, database objects). Section 14 covers the design system and themes, section 15 the security model and section 16 the database. Section 17 lists exactly what was tested and what was not. Section 18 is a practical setup and deployment guide, section 19 describes the demo data and section 20 lists honest limitations. Appendices give a function reference and a file map.

2. Background and goals

2.1 From V1 to V2

CampusDesk V1 is a working production system. It already contains real data in a Supabase project: users, students, teachers, classes, subjects, attendance, marks, fees, notices and more. V2 was designed as an **evolution**, not a rewrite: it keeps V1's tables and column names, keeps V1's feature set, and adds the missing pieces on top. Crucially, V2 can run against the **same Supabase project** as V1, so V1 can stay operational while V2 is tested, and V2 can eventually replace it without any data migration.

2.2 Problems V2 set out to fix

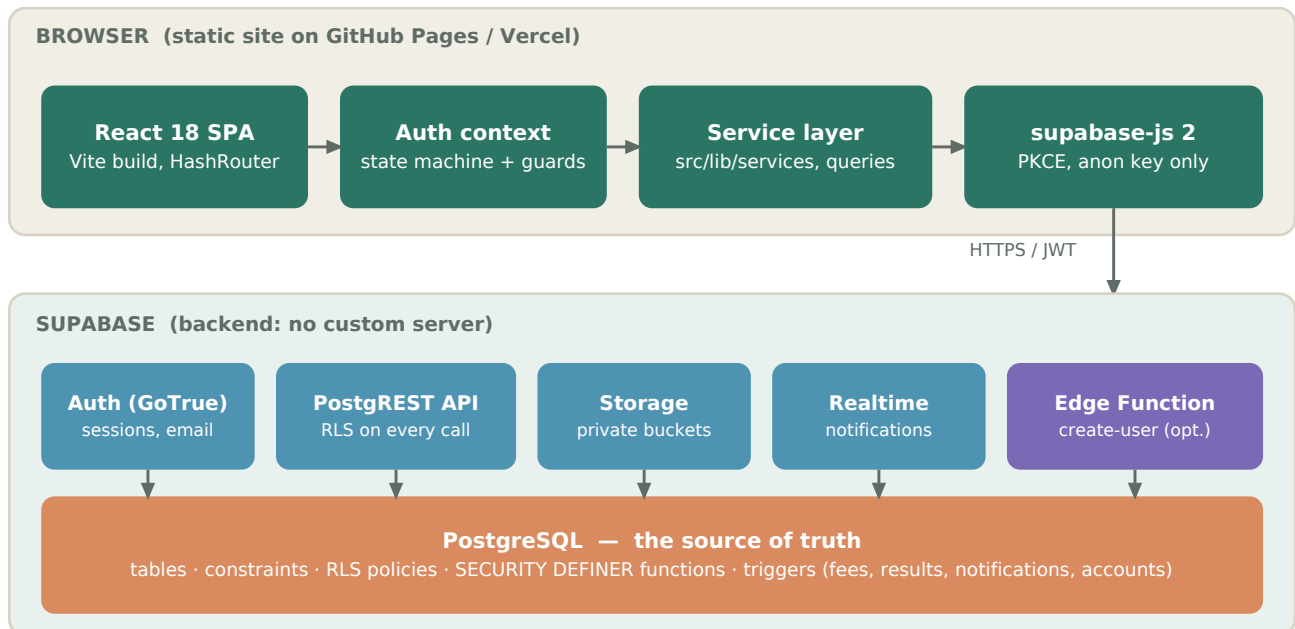
Area	Problem in V1 / early V2	How V2 addresses it
Dashboard	A grid of big navigation cards; no real information	Personalised dashboards with KPIs, today's timetable, charts, notices and an Action-required panel; navigation moved to a sidebar
Registration	Anyone signing up became a student immediately; the signup trigger even trusted a role sent by the browser, so a user could register as admin	Pending → admin approval → role assignment → provisioning → activation; role is never read from the client
Fees	Verification done in the browser (race conditions); students could submit repeated payments; the pay button only checked the remaining balance	Database state machine, advisory locks, unique pending index, atomic verify/reject functions, pay button hidden while a payment is pending
Academic Report	Jumped straight to attendance; no profile, teacher or guardian details	Full professional report with IDs, class teacher, guardian, attendance, marks and indicators
Results	Students with missing marks were published as 0% / grade F	Evaluated vs incomplete detection; review-before-publish; incomplete students get no result
Attendance	Different screens used different formulas	One shared definition in JavaScript and SQL
Security	Profiles readable by every logged-in user; users could change their own role; browser-side account creation	Restrictive RLS for non-active accounts, role-lock trigger, admin-only SECURITY DEFINER functions, optional Edge Function
Look and feel	Single blue-and-white theme	Light/dark/match-device themes with a warm, multi-colour palette

2.3 Design principles

- **The database is the authority.** The browser can request; the database decides.
- **Authentication is not authorization.** Having a login session says who you are, not what you may do.
- **Never destroy V1 data.** All V2 SQL for existing databases is additive and re-runnable. Destructive demo-data scripts live only in a clearly labelled fresh-database folder and refuse to run if real accounts exist.
- **One source of truth for every number.** Fee balances, attendance percentages and grades are computed in one place and reused.
- **Explain, don't expose.** Users see sentences like “A payment for this fee is already awaiting verification”, never raw database errors.
- **Every screen handles loading, empty and error states.** No blank panels, no NaN, no undefined.

3. Architecture

CampusDesk V2 is a classic *serverless single-page application*. The browser downloads a static bundle (HTML, JavaScript, CSS). All dynamic behaviour comes from talking directly to Supabase over HTTPS using the public **anon key**. The anon key is safe to ship in the browser because it grants nothing by itself: every request is evaluated against Row Level Security policies using the signed-in user's identity (a JWT).



3.1 Layers inside the frontend

Layer	Location	Responsibility
Entry & providers	src/index.jsx, src/App.jsx	Mounts ThemeProvider, ToastProvider, AuthProvider, ChildProvider and the HashRouter with all routes
Auth & access	src/auth/accessRules.js, src/context/AuthContext.jsx, src/components/ProtectedRoute.jsx	Account state machine, route decisions (pure, unit-tested), session handling, sign-in/up/out
Layout	src/components/layout/	Sidebar, top bar with breadcrumbs, notification centre, global search, theme toggle
Pages	src/pages/{dashboards,admin,shared,teacher,account,auth}	Screens. Mostly presentation; data comes from services
Services	src/lib/services/	Dashboard data (student, teacher, admin), fees, report, notifications, search, accounts — one function per use case
Queries	src/lib/queries/	Module-level data access inherited from V1 (students, teachers, classes, exams, notices, events, leave, audit, storage)
Shared logic	src/lib/{metrics,dates,format,errors,csv,actions}.js	Attendance policy, fee summaries, IST date logic, formatting, friendly error messages, CSV export, action items
UI kit	src/components/ui/, charts/, dashboard/, timetable/, report/, fees/, accounts/	Cards, badges, modal, toasts, skeletons, empty/error states, charts, timetable grid, report view
Styles	src/styles/theme.css	Design tokens (CSS variables) for light and dark themes plus all component styles

3.2 A request, end to end

Example — a student presses **Pay** on a fee: (1) the *PayDialog* validates the amount and calls `submitFeePayment()` in the fee service; (2) *supabase-js* sends an RPC request to PostgreSQL with the student's JWT; (3) PostgreSQL runs `submit_fee_payment()`, which checks the caller is the student or their parent and inserts a *pending_verification* row; (4) the `fee_payments_guard` trigger takes an advisory lock for that student+fee, re-computes the confirmed balance, and refuses zero, negative, excessive or duplicate requests; (5) a notification trigger tells the admins and the student; (6) the browser reloads the fee lines through `get_fee_summary()` and the Pay button disappears. Every rule in steps 3-5 runs even if someone bypasses the React UI and calls the API directly.

4. Technology stack and project structure

Technology	Version	Why it was chosen
React	18.3	Component model; large ecosystem; matches V1
Vite	5.4	Fast dev server and static build; supports REACT_APP_* and VITE_* variables
React Router	6.30	HashRouter keeps routes working on static hosting after a refresh (URLs look like /#/timetable)
Recharts	2.15	Responsive SVG charts; theme-aware colours resolved at render time
supabase-js	2.45	Auth, database API, storage, realtime; configured with the PKCE flow
Vitest + jsdom	2.1 / 25	Unit and component tests without a browser
PostgreSQL (Supabase)	15/16	RLS, SECURITY DEFINER functions, triggers, advisory locks, jsonb
Fonts	—	Plus Jakarta Sans (interface) and Fraunces (headings) from Google Fonts, with system fallbacks

4.1 Repository layout

```

CampusDesk-V2/
├─ index.html           applies saved theme before first paint
├─ vite.config.js       base path, env prefixes, chunk splitting, test config
├─ package.json         scripts: start, build, test, test:sql, deploy
├─ .env.example         REACT_APP_SUPABASE_URL / _ANON_KEY (+ optional flags)
├─ .github/workflows/deploy.yml  GitHub Pages
├─ src/
│  ├─ auth/accessRules.js  pure access rules (unit-tested)
│  ├─ config/navigation.js  single source of truth for menus + breadcrumbs
│  ├─ context/             AuthContext, ChildContext, ThemeContext
│  ├─ components/          layout, ui, charts, dashboard, timetable, report, fees...
│  ├─ hooks/useAsync.js    loading / error / reload wrapper
│  ├─ lib/                 services/, queries/, metrics, dates, errors...
│  ├─ pages/               dashboards, admin, shared, teacher, account, auth
│  └─ styles/theme.css     design tokens + components
├─ supabase/
│  ├─ migrations/          001 ... 008 (+ 900 superseded) → V1 database
│  ├─ v2-fresh/            schema ... seed → NEW database only
│  └─ functions/create-user/ optional Edge Function
│     └─ audit/rls_audit.sql read-only policy review
├─ tests/                unit/, sql/, integration/
├─ scripts/build-fresh.py generates v2-fresh/* from the migrations
└─ docs/                 AUTH_ACCOUNT_MANAGEMENT, DATABASE_SETUP, TESTING, ...

```

5. Roles, navigation and routing

CampusDesk has four roles. The role determines the sidebar, the dashboard and which routes may be opened. Menus are defined once in `src/config/navigation.js`; the same configuration drives the sidebar, the mobile drawer and the breadcrumbs, so they can never disagree.

Role	Sidebar structure	Typical purpose
Student	Dashboard · Academics (Timetable, Attendance, Results, Academic Report) · Finance (Fees, Payment History) · Communication (Notices, Events, Chat) · Leave · Documents	See today's classes, performance, fees and what needs attention
Teacher	Dashboard · Teaching (My Classes, Attendance, Marks, Academic Reports) · Schedule (My Timetable) · Communication (Notices, Events, Student Queries) · Leave · Documents	Run classes, mark attendance, enter marks, answer queries
Parent	Dashboard · Child (Attendance, Results, Academic Report, Timetable) · Finance (Fees, Payment History) · Communication (Notices, Events, Chat) · Documents	Follow one or more children; pay and track fees
Admin	Dashboard · People (Account Requests, Students, Teachers, Parents) · Academics (Classes & Subjects, Timetable, Exams & Results) · Finance (Fees & Payments) · Communication (Notices, Events) · Operations (Leave Requests, Audit Logs) · Documents	Approve accounts, manage the institution, verify payments, publish results

5.1 The shell

- **Sidebar** — grouped navigation; each section has its own accent colour; collapsible on desktop (the choice is remembered); becomes a slide-in drawer on phones. Account links (My Profile, Settings, Logout) sit at the bottom with the user's name and role.
- **Top bar** — breadcrumbs, the theme toggle, a role-aware global search and the notification bell with an unread badge. For parents the bar always shows *which child* is being viewed.
- **Skip link** and keyboard-friendly menus, focus outlines and ARIA labels throughout.

5.2 Route protection

Every application route is wrapped in `ProtectedRoute`, which asks a pure function (`resolveAccess`) what to do. The order of checks is deliberate: loading → not logged in → account-state pages → wrong role → allow. Because the function is pure, the whole decision table is covered by unit tests. See section 6.5 for the state pages.

Group	Routes
Public	<code>/login</code> , <code>/register</code> (<code>/signup</code> redirects), <code>/forgot-password</code> , <code>/reset-password</code>
Account state	<code>/pending-approval</code> , <code>/registration-rejected</code> , <code>/account-suspended</code> (<code>/suspended</code> redirects), <code>/account-setup-required</code> , <code>/unauthorized</code>
Everyone (active)	<code>/</code> (<code>/role</code> dashboard), <code>/profile</code> , <code>/settings</code> , <code>/notifications</code> , <code>/timetable</code> , <code>/attendance</code> , <code>/results</code> , <code>/report</code> , <code>/notices</code> , <code>/events</code> , <code>/documents</code>
Student / Parent / Teacher	<code>/chat</code> , <code>/leave</code> (admin, teacher, student), <code>/fees</code> and <code>/payment-history</code> (student, parent; admin sees the fee console at <code>/fees</code>)
Teacher	<code>/my-classes</code>
Admin only	<code>/account-requests</code> , <code>/students</code> (+ <code>/new</code> , <code>/import</code> , <code>/:id</code> , <code>/:id/report</code>), <code>/teachers</code> (+ <code>/new</code> , <code>/import</code> , <code>/:id</code> , <code>/:id/report</code>), <code>/parents</code> , <code>/classes</code> , <code>/audit-log</code>

6. Authentication and account management

This is the part of V2 that changed most recently, and it is built around one distinction: **who you are** versus **what you are allowed to be**. Supabase Auth answers the first question (a session exists for a verified email and password). CampusDesk's own tables answer the second (a profile with an assigned role, a lifecycle status, and — for students and teachers — an entity record). A login session alone never opens the application.

6.1 The building blocks

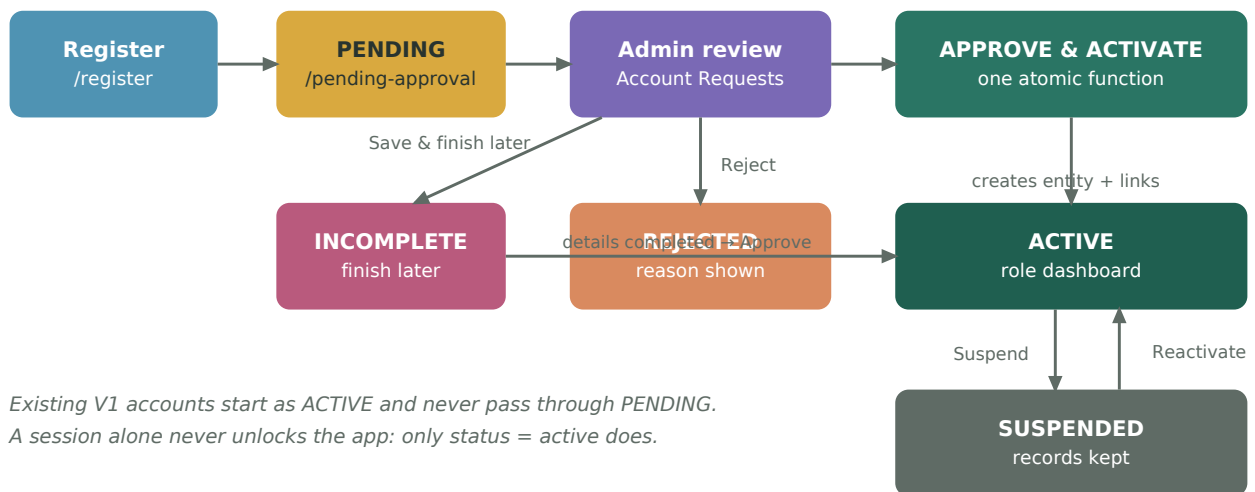
Concept	Where it lives	Meaning
Authentication account	auth.users (Supabase)	Email + password identity. Created by registration.
Profile	public.profiles (id = auth user id)	Name, email, phone, avatar, role , status , requested_role , decision fields
Requested role	profiles.requested_role	What the applicant asked for (student / parent / teacher). Only a request; never grants access.
Assigned role	profiles.role	What an administrator decided. Meaningful only while status = active.
Entity	students / teachers (id = profile id); parent link = students.parent_id	The institutional record: roll number, class, department, children, subjects...
Lifecycle status	profiles.status	pending · active · incomplete · suspended · rejected

6.2 Account states

State	Meaning	What the user sees	Data access
pending	Registered, waiting for an administrator	/pending-approval — “Your account is waiting for administrator approval” with email, requested role and status; Check status / Log out	None
active	Approved and provisioned	The dashboard for the assigned role	Role-based, via RLS
incomplete	Approved, but required entity details are missing (or an active student/teacher has no record)	/account-setup-required	None until completed
suspended	Disabled by an administrator; history preserved	/account-suspended with the administrator's note	None
rejected	Registration refused	/registration-rejected with the reason	None

An unexpected status value or an invalid role is treated as *incomplete* — the safe default — never as an accidental role. An authenticated user with *no profile at all* sees a clear “profile not provisioned” message, not a generic error.

6.3 The lifecycle



6.4 Registration screen

/register (the old **/signup** redirects here) collects: full name, email, optional phone, “I’m registering as” (Student, Parent / guardian or Teacher), password and confirm password. A line under the dropdown states plainly that this is only a **request** that an administrator will review. Validation covers name, email format, an 8-character minimum password and password match before anything is sent.

What is sent to Supabase is deliberately small: the user metadata contains `full_name`, `phone` and `requested_role` — never a role or status. The client also sanitises the request: any value other than student, parent or teacher (for example a tampered *admin*) is replaced by null before sending, and the database ignores it again. After success the user is taken to **/pending-approval** (with a note to confirm their email if Supabase email confirmation is enabled), never to a dashboard.

6.5 Account-state pages

Five pages share one calm layout (AccountLayout: icon, title, explanation, key details, actions) and respect the active theme. Each is wrapped in `StatusRoute`, so it can only be seen by people actually in that state — an active user opening **/pending-approval** is sent to the dashboard; a suspended user is sent to **/account-suspended**.

Page	Shown when	Content
/pending-approval	<code>status = pending</code>	Registered email, requested account, status, Check status, Log out. Also viewable right after registering when email confirmation means no session yet.
/registration-rejected	<code>status = rejected</code>	“Your CampusDesk registration was not approved”, the administrator’s reason if provided, Back to login.
/account-suspended	<code>status = suspended</code>	Explanation, the administrator’s note, Log out.
/account-setup-required	<code>status = incomplete</code>	Explains an administrator must finish the details; Check again, Log out.
/unauthorized	active account, wrong role	“You don’t have access to this page” with Back to my dashboard and Log out. Used only for this case.

6.6 Login decision tree

Situation	Result
Wrong email or password	"Invalid email or password." (no hint about which part was wrong)
Email not confirmed / network error / too many attempts	A specific, friendly message
Authenticated, no profile	Signed out again and told: "Your account has been created but your CampusDesk profile has not been provisioned yet. Please contact an administrator."
pending / rejected / suspended / incomplete	Redirected straight to the matching page
active	To the page they were heading to, or the dashboard for their role (student, teacher, parent, admin)

The login form offers "Keep me signed in on this device". When unticked, the session ends when the browser session ends. Demo-account shortcut buttons appear on the login page for convenience.

6.7 AuthContext and session handling

AuthContext exposes a single state value: *loading*, *unauthenticated*, *error*, *no_profile*, *pending*, *rejected*, *suspended*, *incomplete* or *active*, plus *role* — which is **null unless the account is active**. Design points that remove the classic authentication bugs:

- **No flash and no premature redirect.** While the session or account is loading the state is *loading*; guards show a spinner instead of redirecting to */unauthorized*.
- **One load at a time.** A sequence counter discards stale account loads (for example a sign-out arriving while a load is in flight), and duplicate loads triggered by auth events are de-duplicated.
- **One account query.** The database function `my_account_state()` returns status, role, requested role, reason fields and whether the entity record exists, in a single round trip. If migration 008 is not installed yet, the context falls back to reading the profile row, so the app keeps working during rollout.
- **Connection problems are not confused with missing accounts.** A failed load shows a "We couldn't load your account — Try again" screen.
- **Sign-out clears everything** (account, profile, recovery flag, session marker).

6.8 Administrator workflow: Account Requests

Admin → People → Account Requests (`/account-requests`) is the control centre for accounts. It opens on the Pending tab so new registrations are the first thing an administrator sees, and the admin dashboard shows a pending counter and an Action-required line that links here. A notification ("New registration request — X registered and requested a student account") is created automatically by the database when someone registers.

- **KPIs:** total users, pending approvals, active students, active teachers, active parents, suspended.
- **Filters:** status tabs with counts (All, Pending, Active, Incomplete, Rejected, Suspended), a role filter, and a debounced search over name, email, phone, student ID, roll number, teacher ID and employee ID; paginated 25 per page.
- **Table:** avatar (colour from the name), name, email and phone, role (or "Requested: Student" for unapproved applicants), status badge, record code and detail (e.g. STU-2026-004 · Class

10-A), registration date. Pending rows are tinted so they stand out.

- **Row actions:** Review (pending / incomplete / rejected), Edit (active), Suspend (with a note), Reactivate, Make admin. Nobody can act on their own row.

6.9 The review and provisioning dialog

Review opens a dialog with the applicant's personal information, registration information (requested role, date, status), and a **Role assignment** dropdown containing only Student, Parent and Teacher — the requested role is marked “(requested)” but the administrator may choose differently. The role-specific form follows:

Role	Fields collected	What the database creates
Student	Roll number*, class*, parent/guardian, date of birth, gender, admission date, address (Student ID is generated automatically; section and academic year are read from the class)	A <i>students</i> row with the same id as the profile, an automatic student code like STU-2026-004 , the parent link
Parent	Children* — a searchable checklist of students (those linked to another parent are disabled)	<i>students.parent_id</i> set for each selected child; children removed from the list are unlinked
Teacher	Employee ID*, department*, qualification, joining date, classes and subjects taught (checkboxes grouped by class), class teacher of (classes)	A <i>teachers</i> row with an automatic teacher code like TCH-2026-003 , <i>teacher_subjects</i> rows, <i>classes.class_teacher_id</i>

* required to activate.

Buttons: **Approve & activate**, **Save & finish later** (saves what was entered and marks the account *incomplete*), **Reject** (optional reason shown to the applicant) and **Cancel** (the account simply stays pending). All of this is one call to `admin_provision_account()`, a single database transaction — if any step fails, nothing is changed, so there are no half-provisioned accounts.

6.10 Safety rules inside the provisioning functions

- Only an **active administrator** may call them; the check is inside the function, and EXECUTE is revoked from public.
- Administrators cannot modify themselves, and existing administrators can only be suspended or reactivated (never converted by provisioning).
- **Administrator creation is a separate, explicit act:** `admin_promote_to_admin()` promotes an *active* account; the public form can never request it. A suspended admin immediately stops being an admin (`is_admin()` requires `status = active`). Because the acting administrator must themselves be active, the last active administrator can never be suspended through the application.
- A role cannot be changed once the account already has records under it (attendance, marks, fees for students; assignments, timetable or class-teacher duty for teachers; children for parents).
- A parent cannot take a child who belongs to another parent; roll numbers and employee IDs must be unique; a teaching assignment must reference a subject that belongs to the chosen class; a class that already has a different class teacher cannot be silently taken over.
- Every step writes to the existing audit log and, where relevant, notifies the applicant (approved, rejected, suspended, reactivated, setup incomplete).

Audit action	Written when
account_registered	Someone registers (written by the signup trigger)

Audit action	Written when
account_approved / account_setup_incomplete	Approve & activate / Save & finish later
role_assigned	A role is assigned or changed
student_provisioned, teacher_provisioned, parent_linked	Entity records are created or children are linked
account_rejected, account_suspended, account_reactivated, admin_promoted	The corresponding admin action

6.11 Password reset

Supabase's default “implicit” flow returns the user with tokens in the URL *fragment* (`#access_token=...`), which collides with a hash router (`#/route`). V2 therefore creates the Supabase client with the **PKCE flow**: email links return as `.../CampusDesk-V2/?code=...` and supabase-js exchanges the code automatically. The forgot-password form sets `redirectTo` to the application's base URL (which must be added to Supabase → Authentication → URL Configuration → Redirect URLs) and remembers that a reset was requested in this browser. When the recovery session appears, the app moves to **/reset-password**, where the user chooses a new password (minimum 8 characters, must match). On success they are signed out and sent to the login page. The page also still accepts `?token_hash=...&type=recovery` links from a customised email template. *Note: PKCE requires opening the link in the same browser that requested it.*

6.12 Other ways accounts are created

- **Edge Function** `create-user` (optional): the administrator's browser calls a Supabase Edge Function; it verifies the caller is an active admin, then uses the service-role key — which exists only inside Supabase — to create the Auth user, profile (already approved) and student/teacher record, rolling the Auth user back if any step fails. Enabled with `REACT_APP_USE_EDGE_FUNCTIONS=true`.
- **Legacy “Add student / teacher / parent” forms** inherited from V1 now create the Auth user and then call the same atomic provisioning function. On a database without migration 008 they fall back to their old steps.
- **Opt-in legacy signup mode**: while V1 is still in use on the same database, an administrator can set `campusdesk_settings.allow_legacy_signup_role = 'true'` so V1's own signup keeps producing active accounts. It can never create an administrator.

6.13 Compatibility with existing V1 users

Existing accounts are untouched: every existing profile stays *active* with its role and relationships, so V1 admins, teachers, students and parents log in exactly as before. Two behaviours change on a shared database and are documented in the migration header: V1's own “Create account” page now produces pending accounts (unless the legacy flag is set), and V1 users who were suspended are now also blocked by the database rather than only by the app. The earlier optional file `900_OPTIONAL_security_hardening.sql` is **superseded** by migration 008 and must not be run after it; it is left unmodified and excluded from the fresh-install build.

7. Dashboards

Every dashboard answers five questions: *what do I need to know, what do I need to do, what is happening today, what is coming next, and how am I performing?* A component that doesn't serve one of them isn't on the dashboard. KPI cards are informational, not navigation cards; each links to a detailed page through the sidebar or a small “View all” link.

7.1 Student dashboard

- **Greeting and identity** — “Good morning, Aarav” with class and roll number and today's date (Indian format).
- **Action required** (hidden when empty) — attendance below 75%, a payment awaiting verification, a rejected payment to resubmit, fees overdue or due within 7 days, an exam within 7 days, leave requests awaiting a decision. Each item links to the right page.
- **KPI cards** — Attendance (with status and week-over-week change), Average score (with lowest subject), Fees (amount due, status, amount pending verification), Pending tasks.
- **Today's timetable** — periods for the student's class on today's weekday (India time). A banner shows **CURRENT CLASS** with minutes left, or **NEXT CLASS** with minutes until it starts; the live period is highlighted, finished periods are dimmed, and gaps of 30 minutes or more render as a *Break* row. The view refreshes itself every 30 seconds.
- **Academic health** — attendance, average score, lowest subject, lowest attendance subject, days to next exam and fee status, each with a text label (never colour alone).
- **Charts** — attendance trend (last 4 weeks), exam performance versus class average, subject performance, subject attendance, and “you versus class average” for the latest exam. Class averages come from an aggregate-only database function and are shown only when at least two students contribute.
- **Upcoming exams, fee snapshot, recent notices, upcoming events.**

7.2 Parent dashboard

Same data and charts as the student dashboard, framed around “How is my child doing?”. The header shows the child's name, class and roll number; KPIs replace “Pending tasks” with “Next exam”. A parent with several children gets a **child switcher** in the top bar; the choice is remembered per parent and every page (attendance, results, report, fees, timetable) follows it. The page *always* states which child is being viewed. A parent sees only the children linked to their account — enforced by the database, not just the interface.

7.3 Teacher dashboard

- **Header** — subjects taught and department; quick actions: Mark Attendance, Enter Marks, Post Notice, Student Queries.
- **KPIs** — classes today, students today, pending attendance (classes already held but not marked), pending marks (started exams with missing marks for the teacher's subjects), open student queries.
- **Today's classes** — each period with time, class, subject, room, number of students and a status badge: Completed, In progress, Attendance pending or Upcoming.
- **Pending academic work** — which exam / class / subject still lacks marks and for how many students.

- **Charts** — class attendance and class average by class and subject, attendance trend across the teacher's classes, performance by exam; plus notices and events.

7.4 Admin dashboard

- **Action required** — new registrations waiting for approval, with a link to Account Requests.
- **KPIs** — pending registrations, students, teachers, classes, overall attendance, fees collected, pending fees (outstanding on a confirmed basis), pending payments, upcoming exams, pending leave.
- **Charts** — attendance by class, academic performance by class, fee collection (collected versus outstanding per class), students and teachers per class.
- **Pending approvals** — leave requests and payments awaiting verification with direct Review / Verify links.
- **Recent activity** — human-readable lines generated from the audit log (“Priya Nair published marks for Mid Term”).
- **System health** — registrations awaiting approval, approved accounts with incomplete setup, students without a parent, students without a class, classes without a class teacher, classes without a timetable, payment requests and leave requests awaiting action, exams with incomplete marks. Items with a count get a Fix link.

7.5 Implementation notes

- Data comes from services (getStudentDashboardData, getTeacherDashboardData, getAdminDashboardData) that issue requests in parallel and return one object; components only present it.
- The student and parent dashboards share one component (StudentOverview) so they cannot drift apart.
- Every section has a skeleton loading state, an empty state with an icon and message, and an error state with a Retry button.
- Numbers are never rendered as NaN, null or undefined; missing percentages display “—”.

8. Timetable

The timetable is stored per class with a weekday (0 = Sunday ... 6 = Saturday), start and end time, subject, teacher and room. The sample data covers Monday-Friday with six periods a day for four classes, with no teacher or room clashes.

8.1 Weekly grid page

- **Students** see their own class; **parents** see the selected child's class; **teachers** see their teaching timetable (class labels instead of teacher names); **admins** pick any class from a dropdown.
- The grid builds its rows from the time slots actually present, so it adapts to any bell schedule. Today's column is tinted, the current period is highlighted with “NOW”, and gaps of 30 minutes or more become BREAK rows. On small screens the grid scrolls horizontally inside its card.
- **Administrators** can add a period (subject, teacher, day, start, end, room) and remove periods through a confirmation dialog; affected students, parents and the teacher are notified automatically.

8.2 Time and conflict rules

- All “today / current / next” logic uses **India Standard Time** (Asia/Kolkata) regardless of the viewer's device time zone, through helper functions in `lib/dates.js`. Dates are shown as *04 Oct 2026* and times as *10:30 AM*.
- **Conflict detection in the database:** a trigger refuses a period that overlaps another period of the same class on the same day, or another period of the same teacher; the interface turns the error into “This class already has a period at that time” or “That teacher is already teaching another class at that time”. Existing rows are never re-checked, so old V1 data is unaffected.

9. Attendance

9.1 One definition everywhere

Attendance policy

`attended = present + late` · `counted = present + late + absent` · approved *leave* is excused and excluded · `percentage = round(100 × attended ÷ counted)`; with nothing counted the value is “—”, never 0% or NaN.

The rule exists twice, deliberately identical: in JavaScript (`pctFromCounts`, `attendancePct` in `lib/metrics.js`) and in SQL (`attendance_percent()`). The student, parent, teacher and admin dashboards, the attendance page, the academic report and the exports all use it. A consistency check run against a seeded database confirmed that for each student the dashboard, report, attendance page and a direct SQL calculation produce the same percentage.

9.2 Student and parent view

The Attendance page shows the overall percentage with the breakdown (present, late, absent, approved leave) and the recent sessions. A note explains that late counts as attended.

9.3 Teacher marking workflow

Teacher → class and subject → date → student roster → mark each student (present, late, absent, leave) → Save. Saving calls the database function `save_attendance()`, which checks the teacher is assigned to that class and subject (or is an admin), refuses future dates, verifies each student belongs to the class, and then updates existing rows or inserts new ones — so saving the same roster twice never creates duplicates. The page confirms with “Attendance saved — 4 new, 0 updated”. Marking a student absent notifies the student and their parent.

9.4 Aggregates for dashboards

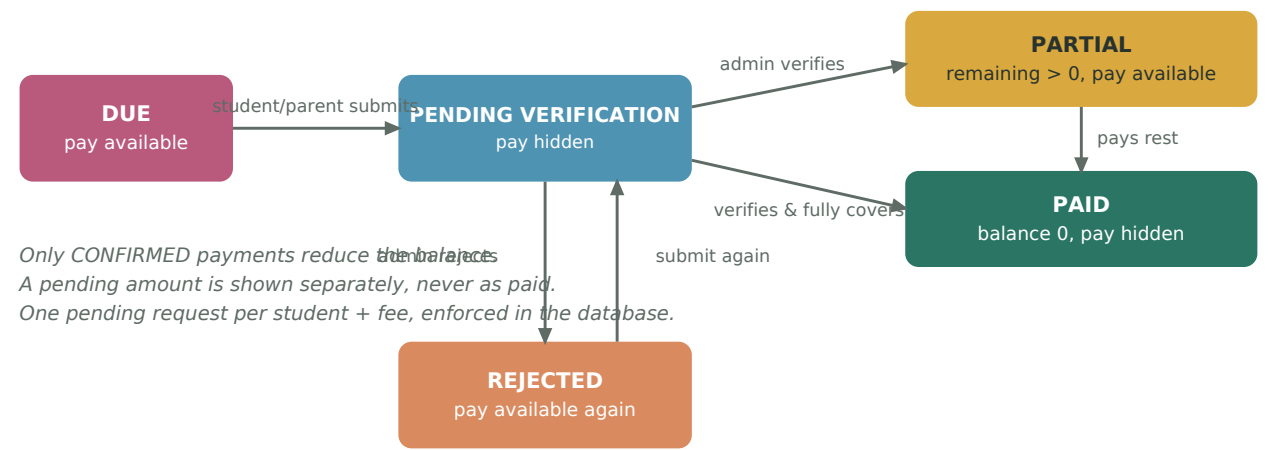
Admin and teacher dashboards do not download thousands of attendance rows.

`attendance_summary_by_class()` returns counts grouped by class and subject (admins see every class; teachers only the classes they teach) and the percentage is derived with the shared rule.

10. Fees and payments

Fees are tracked manually — there is no payment gateway. A student or parent makes the payment with the school (UPI, bank transfer, cash or cheque), records it in CampusDesk, and the school office verifies it. The design goal is that the numbers can never lie: a payment that has not been verified is never treated as paid, a fee can never be overpaid, and two clicks (or two browser tabs) can never produce two pending requests.

10.1 The state machine



Status	Meaning	Pay button
due	Nothing confirmed, nothing pending	Shown
partial	Some confirmed payment; balance remains	Shown (“Pay ₹20,000”)
pending_verification	A submitted payment awaits the office	Hidden — replaced by a disabled “Awaiting verification”
paid	Confirmed payments cover the full fee	Hidden — “Fully paid”
rejected	The last submission was rejected and nothing is pending	Shown again; the rejection reason is displayed

Example: fee ₹50,000, confirmed ₹30,000, pending ₹10,000 → the page shows *Total ₹50,000 · Confirmed paid ₹30,000 · Pending verification ₹10,000 · Remaining ₹20,000* with a notice that includes the submission date, payment mode and reference. It does **not** show ₹40,000 as paid. Once verified, confirmed becomes ₹40,000 and remaining ₹10,000.

10.2 Database enforcement

The fee rules are not React code. They are implemented once in PostgreSQL, and the user interface merely reflects them.

Rule	Mechanism
No zero or negative payment	Guard trigger + table constraint (amount_paid > 0 unless status is due)
No payment larger than the remaining balance	Guard trigger recomputes the confirmed total inside the transaction
No payment once fully paid	Guard trigger

Rule	Mechanism
One pending request per student + fee	Partial unique index <code>uq_fee_one_pending</code> + trigger check
No double verification / editing a decided payment	Paid, partial and rejected rows are immutable; verify/reject accept only <i>pending</i> rows
Concurrent requests are safe	<code>pg_advisory_xact_lock</code> on student+fee serialises competing transactions; verify/reject lock the payment row
Approving after the balance changed	Verification re-checks the balance; “exceeds remaining” blocks approval of a stale pending payment
Only verified payments get receipts	Receipt numbers (RCPT-2026-000123) are issued at verification; <code>get_payment_receipt()</code> refuses pending/rejected payments

Functions: `get_fee_summary(student)` (single source of truth for every fee number), `submit_fee_payment`, `verify_fee_payment`, `reject_fee_payment`, `admin_record_payment` (offline cash/cheque, confirmed immediately), `get_payment_receipt`. Errors use stable codes such as `campusdesk:pending_exists` that the interface translates into plain sentences.

10.3 Student and parent screens

- **Fees** — four summary cards (Total, Confirmed paid, Pending verification, Remaining) and one card per fee item with due date, status badge and the correct action. The **Pay** dialog asks for amount (limited to the remaining balance), payment mode and a reference; submission shows a toast and refreshes immediately.
- **Payment history** — every payment with date, fee, amount, mode, reference, status (rejection reasons shown) and the receipt number of verified payments, which opens a printable receipt.
- The same numbers appear on the dashboard, in Action required, and in notifications — all from `get_fee_summary()`.

10.4 Administrator fee console

Tab	What it does
Verification queue	Pending payments oldest-first with student, fee, amount, mode, reference and date. Approve (confirmation dialog explains a receipt will be issued) or Reject with a reason shown to the student.
Payments	Ledger of all non-due payments with status and date filters, text search, server-side pagination and CSV export; receipts open from the list.
Record payment	Class → student → fee → amount → mode → reference for cash/cheque received at the office; shows a warning if a pending payment exists for that fee.
Fee structure	Add or remove fee items per class (name, amount, due date).

10.5 Receipts

An official receipt shows CampusDesk, the receipt number, payment date, student name, roll number, class and academic year, fee type and total, amount paid, mode, reference, who verified it and when. It can be printed (always in the light theme). Pending and rejected payments never produce a receipt.

11. Exams, marks and results

Marks are entered per student, exam and subject by the teacher of that subject (or by an admin). A result is the student's roll-up for an exam — total, percentage, grade and class rank — and it becomes visible to the student and parent only when it is **published**.

11.1 Grades

Percentage	≥ 90	≥ 80	≥ 70	≥ 60	≥ 50	< 50
Grade	A+	A	B	C	D	F

The same scale exists in SQL (`grade_for()`) and is used by the report and by publishing, so a grade can never differ between screens.

11.2 Missing marks are not zero

In V1, publishing computed results for every student who had *any* marks — so a student whose Mathematics mark had not been entered yet could be published as a low percentage and grade F. In V2 a student is **evaluated** only when marks exist for *every* subject of the exam's class; otherwise they are **incomplete** and receive no result at all (the student sees nothing for that exam rather than a false failure).

11.3 Review before publishing

In Admin → Exams & Results, **Review & publish** calls `exam_result_status()` and shows a summary dialog:

Students	Evaluated	Incomplete	Average	Highest	Lowest
e.g. 24	e.g. 21	e.g. 3	e.g. 82.4%	e.g. 96%	e.g. 51%

If any student is incomplete, the Publish button stays disabled until the administrator ticks “*Publish anyway — these students will show ‘not evaluated’ and get no result*”. The database enforces the same rule: `publish_exam_results(exam, allow_incomplete)` raises `incomplete_results` unless the flag is true. Ranks are computed among evaluated students only, results already published for incomplete students are hidden again, and the action is written to the audit log. Teachers and students calling the function are refused. `unpublish_exam_results()` hides an exam's results again.

11.4 Who sees what

- Students and parents see marks and results only for **published** exams (RLS on *marks* and *results*, and the report function).
- Teachers can read and enter marks for subjects they teach; they cannot edit another teacher's subject.
- Class averages for the “you versus class” charts come from `class_exam_averages()`: aggregates only, only published results, only for a class the caller belongs to, and only when at least two students contribute — so no individual classmate's score can be inferred.

12. Academic Report

The report is a printable, professional document generated from one database call, `get_student_report(student)`, which returns a JSON document containing everything below. The function checks that the caller is the student, their parent, a teacher of their class or an administrator; students and parents receive published results only.

Section	Contents
Header	Institution name, logo placeholder, "Student Academic Report", academic year
Student	Name, Student ID (STU-2026-001 — never a database UUID), roll number, class, section, academic year, email, phone, date of birth, gender
Class teacher	Name, Teacher ID, email, phone
Parent / guardian	Name, email, phone
Academic summary	Overall attendance (with status), average score (with status), best subject, subject needing attention, fee status
Attendance	Per subject: present, late, absent, leave, total, percentage; overall row; the policy line explaining how the percentage is calculated
Academic performance	Subject, exam, marks, maximum, percentage, grade for every published mark
Exam-wise results	Exam, term, total, percentage, grade, class rank
Subject-wise performance	Average per subject with an indicator (Excellent, Good, Fair, Needs work)
Footer	Generation time and the note that only published results are included

- **Print / Save as PDF** uses a dedicated print stylesheet: sidebar, top bar and buttons are hidden, A4 page with margins, tables avoid breaking across pages, and the page prints in the light theme even if the app is in dark mode.
- **Export marks (CSV)** downloads Subject, Exam, Marks, Maximum, Percentage, Grade.
- Students open it from Academics → Academic Report; parents from Child → Academic Report (following the child switcher); administrators from a student's detail page; teachers keep their class-level teacher report.

Student and teacher codes are generated by triggers (STU-YYYY-NNN, TCH-YYYY-NNN) from the admission/joining year and a sequence; existing V1 rows were back-filled without overwriting anything.

13. Other modules

13.1 Notifications

A bell in the top bar shows an unread badge. Clicking it opens a panel with an icon per type (notice, result, fee, timetable, event, leave, query, attendance, account), title, body, relative time (“10 minutes ago”), an unread dot, click-to-open links, **mark as read** and **mark all as read**. A full Notifications page lists up to 100. New notifications appear instantly through Supabase Realtime and the panel refreshes whenever it is opened.

Notifications are created only by database triggers (SECURITY DEFINER), so users cannot forge them, and users may only flip *is_read* on their own rows. Triggers cover: new notices (to the targeted audience), published results, fee submissions (to admins and the student), verification and rejection, leave requests and decisions, timetable changes, new events, student queries and teacher replies, absences, and the account events of section 6.

13.2 Global search

A search box in the top bar (inside the drawer on phones) is role-aware and debounced: administrators search students, teachers, parents, classes and subjects; teachers search the classes and students they teach; students and parents search notices, events and subjects (parents also their child). Every query still passes through RLS, so a result can never be a record the user may not read.

13.3 Notices and events

Notices can target everyone, students, parents, teachers or a single class; they can be pinned and have an expiry date. Events have a title, date, optional time, location, description and a type (exam, meeting, activity, deadline, holiday, other) shown as a coloured badge. Both appear on dashboards and have full pages. Row Level Security filters notices by role and class, so each person sees only what is addressed to them.

13.4 Leave

Students and teachers request leave with dates and a reason; teachers decide on their students' requests and administrators on everything. The database refuses an end date before the start, requests that start more than 30 days in the past, and a request overlapping another pending or approved request of the same person. The approver, decision time and (columns for) leave type and rejection reason are stored; applicants are notified of the decision.

13.5 Teacher queries (chat)

Students and parents send questions to a teacher; the teacher replies; both sides are notified. This is the V1 chat retained as a question-and-answer flow. Threaded follow-ups are listed as a limitation in section 20.

13.6 Documents

Retained from V1: users upload documents (ID, certificate, academic, fee, other) into a **private** storage bucket under their own folder and open them through short-lived **signed URLs**. Storage policies check that the first folder name equals the user's id; administrators can manage all documents.

13.7 Profile and settings

The profile page shows photo, name, role, email, phone, status and role-specific details (class and roll number for students, department and employee ID for teachers, children for parents). Users can edit their own name and phone and upload an avatar; email, role and class are managed by administrators. Settings keeps password change and adds an **Appearance** control (Light / Dark / Match device).

13.8 People management (administrators)

- **Students, Teachers, Parents** lists with detail pages; student and teacher detail pages link to the new report view; parents list shows linked children.
- **Add student / Add teacher** forms and **CSV import** pages from V1 (they now provision through the atomic function). The importers do not yet show a validation preview — see limitations.
- **Classes & Subjects** management, **Audit Log** viewer, and CSV export for payments and report marks.

14. Design system and themes

The interface moved from a single blue-and-white look to a warm, calm palette with several soothing accent colours, available in light and dark.

14.1 Palette

Token	Light	Dark	Used for
Background / surface	#F4F1EA / #FFDF8	#141817 / #1C2220	Page and cards (warm paper, soft charcoal)
Primary (sage teal)	#2A7564	#6CC5AC	Main actions, active navigation
Terracotta	#D98A5F	#EBA07A	Charts, KPI accents, danger-leaning highlights
Honey	#D9A93F	#E6C05F	Warnings, charts
Lavender	#7A69B5	#A99BE0	Teachers, meetings, charts
Sky	#4F93B3	#78B8D8	Information, attendance charts
Rose	#B95A7D	#E08AAB	Pending items, exams, admin badge
Success / Warning / Danger / Info	#376F3D / #8F5D0F / #B04535 / #2F6F8F	lighter equivalents	Status badges and notices (always with text)

- All colours live in two blocks at the top of `src/styles/theme.css`; no component hardcodes a colour. Contrast was checked against WCAG AA in both themes (body text $\geq 7:1$; muted text, status text and primary-on-surface $\geq 4.5:1$) and one green was darkened after the check.
- Typography: Plus Jakarta Sans for the interface and Fraunces (a serif) for headings and KPI values, giving the product a more editorial, less generic feel.
- Multi-colour touches that help scanning: KPI cards have a coloured top border, each sidebar section has its own icon colour, avatars take one of six soft colours from the person's name, event types have distinct badges.

14.2 Theme behaviour

- A toggle (sun/moon) is in the top bar and on the login/register pages; Settings → Appearance offers Light, Dark and **Match device**, which follows the operating system live.
- The choice is stored in the browser; a tiny script in `index.html` applies it *before* the first paint, so dark mode never flashes white.
- Charts resolve colours from the active theme at render time (SVG attributes cannot use CSS variables reliably) and redraw when the theme changes.
- Printing is always light: the theme provider switches to the light palette for the print event and restores it afterwards.
- The browser's `theme-color` meta tag follows the theme.

14.3 Components and states

Reusable pieces: Card, StatCard, Badge, Modal (focus management, Escape to close), ConfirmDialog for destructive actions, Toast notifications, Skeleton loaders, EmptyState and ErrorState, Icon set, charts (line, bar, comparison, stacked fee chart, distribution, donut), timetable grid, report view, receipt dialog. Accessibility basics: semantic buttons and labels, ARIA roles on tabs, dialogs, menus and progress bars, visible focus, a skip-to-content link, status always communicated with text as well as colour, and reduced-motion support.

15. Security model

The security posture is **defence in depth with the database as the final authority**. Route guards and hidden buttons are conveniences; a malicious client calling the API directly meets the same rules.

15.1 What a malicious user cannot do

Attack	Why it fails
Register as admin / teacher by sending a role in signup metadata	The signup trigger ignores any client role; accounts are created <i>pending</i> ; requested role is limited to student, parent, teacher and is only a request
Change own role or status with a direct API call	<code>guard_profile_update</code> raises <i>role_locked</i> for non-admins (id, role, status, requested_role and decision fields)
Insert a profile or an entity row for themselves	The trigger refuses profile inserts; students/teachers tables have no self-insert policy; pending accounts are blocked by restrictive RLS
Use the app while pending, rejected, suspended or incomplete	Restrictive policy <code>v2_require_active_account</code> on every application table; <code>get_my_role()</code> returns null; profiles readable only for the own row
Call admin functions (approve, publish, verify, promote)	Each function checks <code>is_admin()</code> (active administrator) internally; EXECUTE revoked from public
Read another student's attendance, marks, fees or report	RLS by ownership and relationships; report/fee functions check student, parent, teacher-of-class or admin
A parent reading another family's child	Access only through <code>students.parent_id</code>
A teacher editing another teacher's subject or reading fees	Marks/attendance policies require a <code>teacher_subjects</code> assignment; fee tables have no teacher policy
Approve own payment, or pay twice, or overpay	Fee functions and guard trigger (section 10)
Publish results	<code>publish_exam_results</code> is admin-only; students cannot write the results table
Read private documents of others	Storage policies check the owner folder
Forge notifications	Users may only update <code>is_read</code> on their own rows; inserts come from SECURITY DEFINER triggers
Steal the service-role key	It never appears in the frontend or the bundle; only the optional Edge Function (server-side) uses it

15.2 Findings fixed in the existing database

- V1's signup trigger trusted `raw_user_meta_data.role` (privilege escalation through the public Auth API).
- V1's "authenticated users read all profiles" policy exposed everyone's email and phone to any logged-in account, including unapproved ones.
- V1's "user updates own profile" policy was not limited by column, so users could change their own role/status.
- Fee verification happened in the browser and could race; results published missing marks as failures.

15.3 Implementation details

- Privileged functions are SECURITY DEFINER with a fixed `search_path` and explicit `auth.uid()` checks.
- RLS is never disabled and no sensitive table uses a blanket `USING (true)` policy. Restrictive policies are AND-ed with V1's existing permissive ones, so existing rules are preserved and tightened.
- Friendly error mapping hides database internals from ordinary users; raw errors are logged only in development.

- CSV exports neutralise spreadsheet formula injection (cells beginning with = + - @ are prefixed).
- Only the anon key is used in the browser (REACT_APP_SUPABASE_ANON_KEY).

16. Database reference

V1 and V2 use the same core tables and columns, so V2 can run on the V1 project. Changes ship as numbered, additive, idempotent migrations. They contain no DROP TABLE or TRUNCATE, never delete existing rows, and validate only *new* writes (existing V1 rows are never re-checked).

16.1 Migrations

#	Name	Adds
001	base_compatibility	notifications table; extra nullable columns (timetable.room; events times/type/audience; leave type/rejection reason/decision time); fee status check widened to five values; RLS helper functions (get_my_role, is_admin, in_class, teaches_student, is_parent_of, can_view_profile); notification RLS
002	fee_integrity	Fee columns (reference_note, submitted_by, verified_by, verified_at, rejection_reason); receipt sequence; unique pending index (skipped with a warning if V1 data already violates it); guard trigger; get_fee_summary, submit/verify/reject_fee_payment, admin_record_payment, get_payment_receipt
003	report_support	student_code / teacher_code (+ triggers and back-fill); attendance_percent, grade_for; get_student_report
004	results_integrity	exam_result_status, publish_exam_results, unpublish_exam_results, class_exam_averages
005	notifications	Notification and audit triggers for notices, results, fees, leave, timetable, events, queries and absences; realtime publication
006	leave_attendance_timetable	Leave date/overlap guard; timetable conflict guard; save_attendance; attendance_summary_by_class
007	indexes	Performance indexes (create index if not exists)
008	auth_account_management	Account states and requested role; secure signup trigger; profile guard; restrictive RLS; settings table; account functions (section 6)
900	OPTIONAL hardening	Superseded by 008 — do not run after 008. Left unmodified for history; excluded from the fresh build

16.2 Two install paths — never mixed

	Option A — existing V1 project	Option B — brand-new project
SQL	supabase/migrations/001 → 008, in order	supabase/v2-fresh/: schema → functions → triggers → rls → storage → seed
Data	All V1 data preserved	Demo data only
Safety	Additive and re-runnable. Back up first and try on a copy	FRESH DATABASE ONLY. The seed refuses to run if any real (non-demo) account exists; a separate reset script is guarded the same way
Generated files	—	schema/functions/triggers/rls/storage are generated from the migrations by scripts/build-fresh.py so the two paths cannot drift

Verified against V1's SQL, not your live database

The migrations were validated on a database built from V1's own SQL files and re-applied twice; existing rows survived and the fee flow worked. The live V1 project was never accessed. Run the migrations on a copy (or a Supabase branch) first, and use `supabase/audit/rls_audit.sql` (read-only) to review live policies.

16.3 Main tables

Table	Purpose / key columns
profiles	id → auth.users; role, status, requested_role, full_name, email, phone, avatar_url, decided_by/at, rejection_reason, status_reason
students / teachers	Entity records keyed by the profile id: roll_no, class_id, parent_id, student_code · employee_id, department, teacher_code
classes, subjects, teacher_subjects	Class (name, section, academic year, class teacher), subjects per class, teacher ↔ class ↔ subject assignments
timetable	class, subject, teacher, day_of_week, start/end time, room (+ conflict trigger)
attendance	student, class, subject, date, status (present/late/absent/leave), marked_by
exams, marks, results	Exams per class; marks per student/exam/subject; published results with total, percentage, grade, rank
fee_structure, fee_payments	Fee items per class; payments with status, mode, reference, receipt number, verifier and timestamps
notices, events, leave_requests, teacher_queries	Communication and requests
notifications, audit_logs	Per-user notifications; audit trail of important actions
campusdesk_settings	Admin-only switches (e.g. allow_legacy_signup_role)

17. Testing and quality

Testing is layered: fast unit/UI tests for logic and screens, database tests for the rules that matter most (run against a real PostgreSQL), and a service-level run through PostgREST with real JWTs.

Layer	Command / location	Count	Covers
Unit & UI	npm test (Vitest + jsdom)	66	Attendance policy, fee model, Pay-button cases rendered through the real component, report derivation, action items, error messages, CSV safety, theme switching, route guards with the real components , login routing for every account state, registration payload (no role sent), logout/login state
Database	npm run test:sql (tests/sql)	173	Fee state machine, overpayment, duplicates, concurrent submissions and concurrent verification , results review/publish, report payload, attendance save, timetable and leave rules, RLS, full account lifecycle (pending, privilege escalation attempts, approve each role, reject, suspend, reactivate, promote, legacy flag, audit)
Service level	tests/integration/account s.test.js	28	Real services + supabase-js + PostgREST + RLS with one JWT per user (27 passed in the run; the 28th assertion had a wrong expected count in the test itself, since corrected)
Consistency	tests/integration/consistency.test.js	—	Same student → same attendance % and fee numbers on dashboard, fees page, report, attendance page and raw SQL
Build	npm run build	—	Production bundle builds successfully

17.1 Test results delivered

Result	Status
npm test	66 passed, 0 failed
npm run test:sql	173 passed, 0 failed
npm run build	Succeeds
Migrations 001–008 applied twice to a database built from V1's SQL	Both runs succeed; existing admin and student stay active; V1 policies kept
Bundle scan	No service-role key or .env in the source or bundle

17.2 What was found by testing

- A global error-mapping pass had turned Login's own friendly messages into “Something went wrong” — caught by the login tests and fixed.
- The student/teacher code triggers lacked SECURITY DEFINER and could hit sequence-permission errors for legitimate inserts — caught by the privilege-escalation tests and fixed in migration 008.
- Several earlier inconsistencies (attendance counted differently across screens; results published with zeros) were found by the audit and removed.

What was NOT tested

Visual rendering and responsive layouts in a real browser (no browser was available); print/PDF output; the real Supabase email flows (confirmation and password reset), Realtime and Storage; the create-user Edge Function; page-refresh session persistence; the deployed GitHub Pages site; and the live V1 database. Please click through each role in both themes once after setup.

18. Setup, deployment and operations

18.1 Local development

```
npm install
cp .env.example .env      # fill in the two values below
npm start                 # http://localhost:5173
npm test                  # unit/UI tests
npm run build              # production build in dist/
```

Variable	Purpose
REACT_APP_SUPABASE_URL	Project URL (Supabase → Project Settings → API). VITE_SUPABASE_URL also works
REACT_APP_SUPABASE_ANON_KEY	Public anon/publishable key. Never the service-role key
VITE_BASE_PATH	Optional. GitHub Pages project path, e.g. /CampusDesk-V2/
REACT_APP_USE_EDGE_FUNCTIONS	Optional. <code>true</code> routes admin account creation through the create-user Edge Function

18.2 Database setup

- **Existing V1 project (Option A):** back up → optionally run `supabase/audit/rls_audit.sql` → run migrations 001 → 008 in order in the SQL Editor → point `.env` at the same project. Do not run anything from `v2-fresh/` and do not run migration 900.

- **Fresh project (Option B):** create a new Supabase project → run v2-fresh/ files in the order schema, functions, triggers, rls, storage, seed.
- If the frontend is deployed before migration 008 runs, it keeps working with the older behaviour; the new account flow switches on as soon as 008 is installed.

18.3 GitHub Pages deployment

- The app uses a hash router, so refreshing any page works on static hosting.
- Add repository secrets REACT_APP_SUPABASE_URL and REACT_APP_SUPABASE_ANON_KEY; set Settings → Pages → Source to GitHub Actions; push to main. The workflow builds with VITE_BASE_PATH=/<repo>/ and publishes dist/.
- In Supabase → Authentication → URL Configuration, set the Site URL and add the Pages URL (e.g. https://aryan-gupta-9044.github.io/CampusDesk-V2/) to the Redirect URLs so password-reset and confirmation links return correctly.
- Vercel also works: framework Vite, build npm run build, output dist, same environment variables.

18.4 First-time operation checklist

- Log in with an existing administrator; confirm the dashboard and Account Requests load.
- Register a test student in a private window; confirm the Pending Approval page; approve it as admin (roll number + class); log in as that student.
- Repeat for a parent (link a child) and a teacher (assign class + subject); reject and suspend one account each.
- Open Fees as a student, submit a payment, confirm Pay disappears; verify it as admin and check the receipt.
- Open the Academic Report and print it; switch themes on each role.

18.5 Optional: Edge Function

```
supabase functions deploy create-user
# then set REACT_APP_USE_EDGE_FUNCTIONS=true and rebuild
```

19. Demo data (fresh database only)

The fresh-install seed creates a complete, relational demo institution for academic year 2026-27. Dates are relative to the day the seed is run, so there is always “this week” data. **Demo credentials are for demonstration only — never use them in production.**

Role	Accounts	Password
Admin	admin@campusdesk.com	Admin@123
Teachers	teacher1 ... teacher6@campusdesk.com	Teacher@123
Students	student1 ... student12@campusdesk.com	Student@123
Parents	parent1 ... parent5@campusdesk.com	Parent@123
Data	Contents	
Structure	4 classes (10-A, 10-B, 11-A, 11-B), subjects per class, 6 teachers with assignments and class teachers	

Data	Contents
Relationships	student1 (Aarav Mehta, 10-A) ↔ parent1; parent2 has two children (shows the child switcher); teacher1 teaches Mathematics in every class
Timetable	120 periods Monday-Friday, six per day, no teacher/room clashes
Attendance	Four weeks of records with realistic variation per student
Exams	Unit Test 1, Mid Term, Unit Test 2 (published) and an upcoming Final Term; 10-B Unit Test 2 is deliberately incomplete to demonstrate review-before-publish
Fees	Every state: paid, partial, due, pending verification, rejected, with receipt numbers and reasons
Other	Notices, events (including an upcoming exam and a holiday), leave requests in every status, teacher queries, notifications (read and unread), audit log

The seed is insert-only (no deletes). It aborts if the database contains any non-demo account or if demo data already exists. Document uploads cannot be seeded because files live in Storage.

20. Known limitations and future work

20.1 Not implemented

Item	Notes
Threaded teacher ↔ student/parent conversations with follow-ups and read receipts	Chat is question-and-answer
Notice attachments; event registration screens	Event registration column exists; no UI
Leave type and rejection-reason entry in the interface	Database columns exist
CSV import preview with duplicate/invalid summary	Importers are V1's and do not yet validate before committing — a bulk importer should not be half-working, so it was left out of the new account flow
CSV export for students and attendance	Payments and report marks can be exported
Admin search by student ID / employee ID / phone in the global search	Account Requests search does support them
Monthly attendance trend and date-range filter	Weekly trend (4 weeks) is available
Attendance-threshold alerts	An absence notification exists; threshold alert does not
Email notifications	Notifications are in-app only
A parent “relationship” label (mother, father...)	The schema links parent and child only through students.parent_id
Browser end-to-end tests	None were run

20.2 Behaviour to be aware of

- On a shared V1 database, V1's own registration and browser-side add-teacher/add-parent flows now create pending accounts (use V2, or the legacy flag while V1 is in use).
- V1 users who were suspended are now blocked in the database, not just by the app.
- Results that V1 already published for students with missing marks are not changed automatically; re-run Review & publish for that exam.
- PKCE reset links must be opened in the browser that requested them.

- Seeding creates users directly in `auth.users`; if Supabase changes its auth schema, create the demo users in the dashboard instead.
- Fonts load from Google Fonts; offline, system fonts are used.

20.3 Suggested next steps

- Run the migrations on a copy of the V1 database and complete the manual checklist in section 18.4 in both themes.
- Add browser end-to-end tests (Playwright) for registration → approval → login and the fee flow.
- Build the threaded messaging module and the import validation preview.
- Add transactional email (approval, rejection, fee receipts) through an Edge Function.

Appendix A — Database function reference

Function	Migration	Who may call	Purpose
get_my_role(), is_admin(), is_active_user()	001/008	RLS helpers	Effective role and admin check; require status = active
in_class, teaches_student, is_parent_of, can_view_profile	001	RLS helpers	Relationship checks used by policies
get_fee_summary(student)	002	student, parent, admin	Per-fee totals, confirmed, pending, remaining, status, can_pay
submit_fee_payment(...)	002	student, parent	Create a pending payment
verify_fee_payment(id) / reject_fee_payment(id, reason)	002	admin	Decide a pending payment (atomic)
admin_record_payment(...)	002	admin	Record an offline confirmed payment + receipt
get_payment_receipt(id)	002	student, parent, admin	Receipt data for a verified payment
attendance_percent(), grade_for()	003	any	Shared formulas
get_student_report(student)	003	student, parent, teacher of class, admin	Complete report document
exam_result_status(exam)	004	admin, class teacher	Evaluated / incomplete summary
publish_exam_results(exam, allow_incomplete) / unpublish_exam_results(exam)	004	admin	Publish or hide results
class_exam_averages(class)	004	class members, admin	Aggregate class averages only
save_attendance(class, subject, date, rows)	006	assigned teacher, admin	Atomic roster save
attendance_summary_by_class(class?)	006	admin, teacher	Counts per class/subject
my_account_state()	008	any logged-in user	Own status, role, requested role, reasons, entity check
admin_account_counts(), admin_list_accounts(...), admin_get_account(id)	008	admin	Account management reads
admin_provision_account(user, role, data, activate)	008	admin	Approve/provision student, parent or teacher atomically
admin_reject_account, admin_suspend_account, admin_reactivate_account	008	admin	Lifecycle actions
admin_promote_to_admin(user)	008	admin	The only way to create an administrator

Appendix B — Important source files

File	What it does
src/auth/accessRules.js	Pure functions: deriveAccount, resolveAccess, resolveStatusPage, landingFor
src/context/AuthContext.jsx	Session + account state machine; signIn, signUp, signOut, refreshAccount
src/components/ProtectedRoute.jsx · account/StatusRoute.jsx	Route guards using accessRules
src/pages/auth/*	Login, Signup (register), ForgotPassword, ResetPassword, AccountStatusPages
src/pages/admin/AccountRequests.jsx · components/accounts/ReviewDialog.jsx	Admin account management UI
src/lib/services/{studentData,teacherData,adminData}.js	Dashboard data assembly
src/lib/services/{fees,report,accounts,notifications,search}.js	Feature services (RPC wrappers)
src/lib/metrics.js · dates.js · format.js · errors.js · actions.js · csv.js	Shared rules, formatting, friendly errors, Action-required builder, CSV
src/pages/shared/Fees.jsx · PaymentHistory.jsx · pages/admin/AdminFees.jsx	Fee screens
src/components/report/StudentReportView.jsx	Academic Report
src/components/timetable/*	Timetable grid, today view, live logic
src/context/ThemeContext.jsx · components/layout/ThemeToggle.jsx · styles/theme.css	Theme system
supabase/migrations/001...008	Database changes for existing V1 databases
supabase/v2-fresh/*	Fresh-install SQL and demo seed
tests/unit · tests/sql · tests/integration	Automated tests
docs/*.md	AUTH_ACCOUNT_MANAGEMENT, DATABASE_SETUP, TESTING, FEATURES, THEME, V1_TO_V2_MIGRATION